FILESCAN: FILE SYSTEM FORENSICS AND AI INTERPRETER

IT 360 FINAL PROJECT REPORT

Zainab Onibudo and Caitlyn Cashion

FileScan is a digital forensics program that simplifies the process of examining file system activities. The primary goal of this project is to automate the collection of file metadata and organize it into a clear, chronological order. This allows investigators to quickly comprehend what happened in a system without individually checking each file. In digital forensics, investigating file systems can be time-consuming and challenging, especially when working with huge directories or erased data. FileScan addresses this issue by scanning a target directory, retrieving crucial file attributes (such as timestamps, permissions, and hashes), and organizing them into a logical order. It also includes file carving to retrieve deleted or hidden files, which is useful when looking into suspect activities.

Another important aspect of this project is the incorporation of AI. Instead of simply providing raw data, FileScan uses an API to provide a plain-English summary of the results. This makes the product more accessible and understandable, particularly for users without strong technical understanding.

FileScan was created with a combination of Bash scripting and Python to make each step of the process more efficient. The core process is handled by a Bash script that loads the target directory, scans the files, extracts metadata, and

organizes everything into structured output folders. It also runs external tools as needed, particularly for file carving. We used standard Linux commands to retrieve metadata such as file names, sizes, permissions, ownership, and timestamps (access, modification, and change times). We also created MD5 hashes to verify file integrity and MIME types to establish file formats. All this data is then used to create a timeline based on modification times, which is exported as a CSV file for simple reading and analysis.

For file carving, we used Foremost to scan raw disk data and retrieve deleted or hidden file parts. This allowed us to look beyond the visible files and uncover more potential evidence that could have gone unnoticed.
The AI component was added using an API. After the scan is finished, the results are transmitted to the API, which provides a plain summary of the findings. This included handling API calls and responses with technologies such as curl and Python to ensure that the data was properly formatted and handled.

For contributions, I worked on integrating the API into the script and managing how scan data is sent and analyzed for the final summary, as well as making the final report. Caitlyn oversaw everything to GitHub and created the parts of the script where the directory chosen is read, and how the script reads it and analyzes and outputs what was discovered. I helped create the script and steps we needed for the demo, and Caitlyn recorded the video and demonstrated how FileScan works. Also, we built and tested everything directly in Visual Studio Code before uploading it to the repository.

The tool successfully created structured forensic reports from the examined folders. Each run generated a new output folder including a timeline.csv file that organizes all file activity chronologically, an SUMMARY_REPORT.txt file with the AI-generated explanation of the findings, and a carved_files directory holding any recovered file pieces. We tested many scenarios to determine how the tool

operated under different settings and data types.

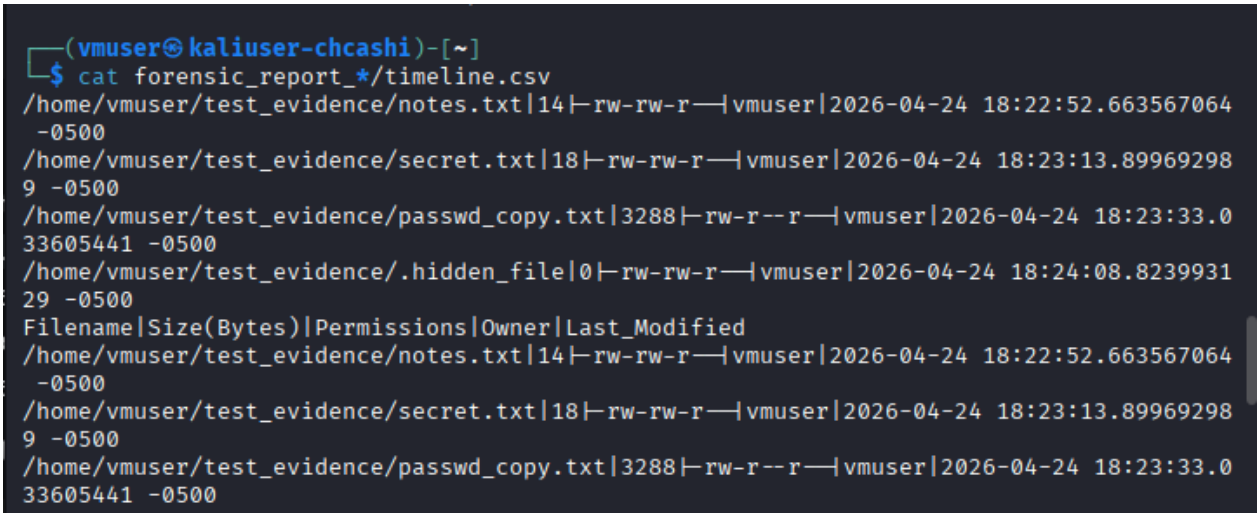
```
┌──(vmuser㉿kaliuser-chcashi)-[~]
└─$ ls forensic_report_*/
forensic_report_20260424_182436/:
carved_files  SUMMARY_REPORT.txt  timeline.csv

forensic_report_20260430_132054/:
carved_files  SUMMARY_REPORT.txt  timeline.csv

forensic_report_20260430_132231/:
carved_files  timeline.csv

forensic_report_20260430_132314/:
carved_files  timeline.csv

forensic_report_20260430_132508/:
carved_files  SUMMARY_REPORT.txt  timeline.csv
```

```
vmuser@kaliuser-chcashi: ~
Session  Actions  Edit  View  Help
Target Directory: /home/vmuser/test_evidence
Scan Date: Thu Apr 30 01:20:54 PM CDT 2026

[AI INVESTIGATOR SUMMARY]

[AI Error] Could not parse response: 'candidates'

[RESOURCES FOUND]

Full Timeline:       forensic_report_20260430_132054/timeline.csv
Carved Files Folder: forensic_report_20260430_132054/carved_files
DIGITAL FORENSICS CASE REPORT
═══════════════════════════
Target Directory: /home/vmuser/test_evidence
Scan Date: Thu Apr 30 01:25:08 PM CDT 2026

[AI INVESTIGATOR SUMMARY]

[AI Error] Could not parse response: Invalid control character at: line 4 column 85 (ch
ar 116)

[RESOURCES FOUND]

Full Timeline:       forensic_report_20260430_132508/timeline.csv
Carved Files Folder: forensic_report_20260430_132508/carved_files

┌──(vmuser㉿kaliuser-chcashi)-[~]
└─$ cat forensic_report_*/timeline.csv
```

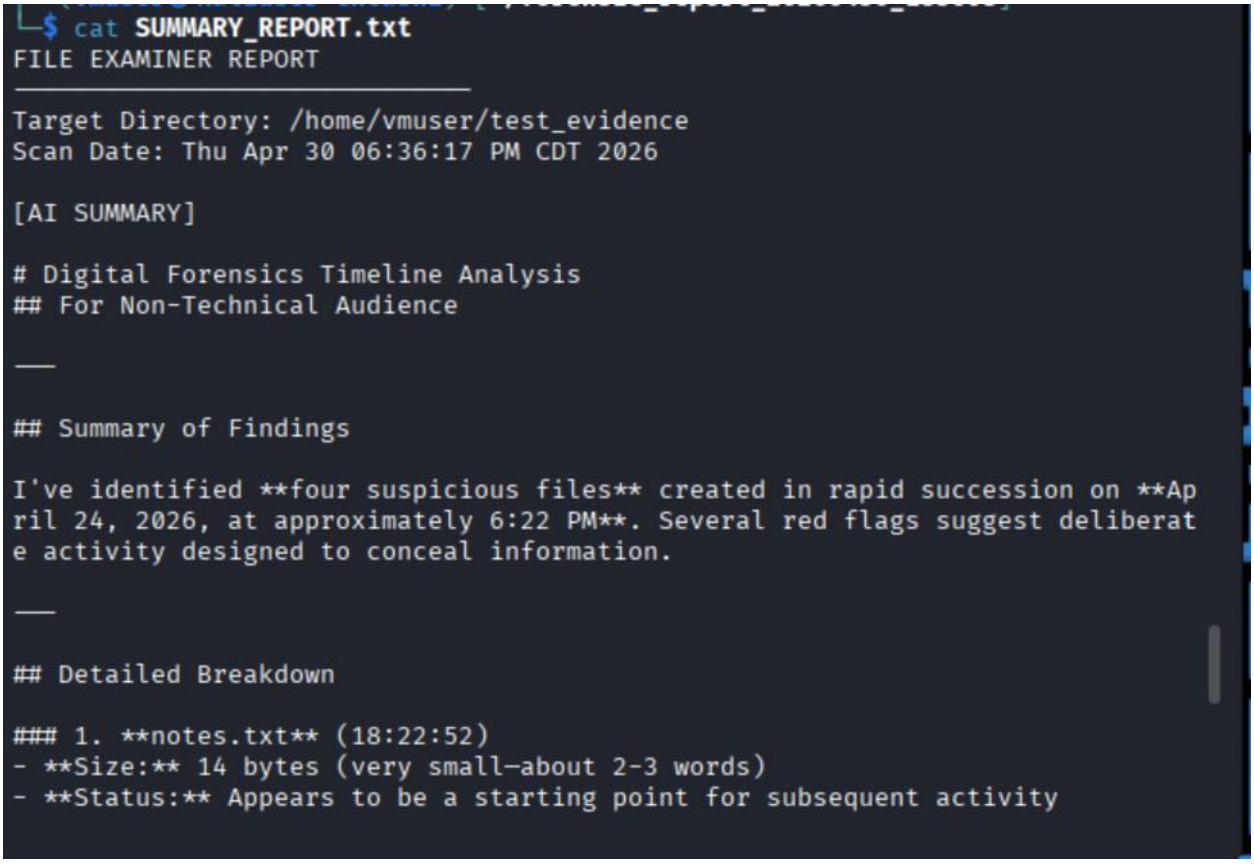
In the first test, fake files were generated in a sample directory. The script generated the timeline and report structure, but the AI summary failed because the API key was not exported correctly.

```
┌──(vmuser㉿kaliuser-chcashi)-[~]
└─$ echo "meeting notes" > ~/test_evidence/notes.txt

┌──(vmuser㉿kaliuser-chcashi)-[~]
└─$ echo "confidential data" > ~/test_evidence/secret.txt

┌──(vmuser㉿kaliuser-chcashi)-[~]
└─$ cp /etc/passwd ~/test_evidence/passwd_copy.txt

┌──(vmuser㉿kaliuser-chcashi)-[~]
└─$ touch ~/test_evidence/.hidden_file

┌──(vmuser㉿kaliuser-chcashi)-[~]
└─$ ./file_examiner.sh ~/test_evidence
[+] Starting Triage on: /home/vmuser/test_evidence
[+] Extracting metadata and building timeline ...
[+] Timeline saved to: forensic_report_20260424_182436/timeline.csv
[+] Running 'foremost' to carve deleted/hidden files ...
[+] Carving complete. See: forensic_report_20260424_182436/carved_files
[+] Sending data to AI for expert interpretation ...
──────────────────────────────────────────────────
[!] ANALYSIS COMPLETE
[!] View the final summary at: forensic_report_20260424_182436/SUMMARY_
REPORT.txt

┌──(vmuser㉿kaliuser-chcashi)-[~]
└─$ ls forensic_report_*/
carved_files  SUMMARY_REPORT.txt  timeline.csv

┌──(vmuser㉿kaliuser-chcashi)-[~]
└─$ cat forensic_report_*/SUMMARY_REPORT.txt
DIGITAL FORENSICS CASE REPORT
═══════════════════════════════
Target Directory: /home/vmuser/test_evidence
Scan Date: Fri Apr 24 06:24:37 PM CDT 2026

[AI INVESTIGATOR SUMMARY]

[AI Error] Could not parse response: 'content'
```

We ran several tests and determined that the problem was with the API and not with the remainder of the script, since the timeline continued to generate correctly.



We eventually solved this by exporting the API key outside of the script rather than hardcoding it. Following that, the AI analysis ran correctly, generating a clear summary report. Overall, the results demonstrated that the tool performs as intended, with both forensic data extraction and AI interpretation working properly after debugging.



During this project, we noticed the importance of debugging and testing, especially when working with APIs. Even though most of the script worked initially, the API issue completely impacted the result, forcing us to troubleshoot and evaluate how we handled the API key. We also discovered that splitting the components (Bash for scanning and Python for AI) made the project easier to manage. It helped us by isolating problems and determining where things were going wrong. Initially, putting the API key directly into the script did not work correctly. This produced problems when the key expired, making troubleshooting more complicated. If we had to do it over, we could develop the tool to include more complicated forensic tools or GUI.

In conclusion, FileScan successfully demonstrates how automation and AI may be integrated to simplify digital forensics. It reduces manual effort, and arranges data

cleanly, and simplifies technical results. This research demonstrates how, with the correct tools, difficult forensic processes may be made more accessible.